



// AI RELIABILITY · ORCHESTRATION REFERENCE

# Agent Orchestration Patterns

Five coordination patterns for multi-agent systems — when to reach for each one, and the specific way each one tends to fail once it's carrying real traffic.

# How to use this

Most agentic systems don't fail because a single agent reasoned poorly. They fail at the seams — the handoff nobody defined, the escalation path that doesn't exist, the human approval gate that was assumed rather than built. This reference walks through five coordination patterns covering most of what a production multi-agent system actually needs, each with a diagram, when it fits, and where it tends to break.

Use it to pick a pattern before you build, or to diagnose which pattern an existing system is unintentionally using — most unplanned agentic systems are accidentally one of these five, just without the tradeoffs having been considered on purpose.

**01 Supervisor / Worker** — one router, several specialists

**02 Sequential Handoff** — a linear pipeline of specialists

**03 Parallel Fan-Out / Fan-In** — split the work, merge the results

**04 Human-in-the-Loop Approval Gate** — pause before anything irreversible

**05 Deterministic-First Escalation** — cheap checks first, model calls only when needed

01

## Match the pattern to the failure mode

Each pattern trades off differently. Pick based on what breaks if it's wrong, not which one sounds most sophisticated.

02

## Patterns compose

A real system is often two of these nested — a supervisor whose workers each run a sequential handoff, for instance.

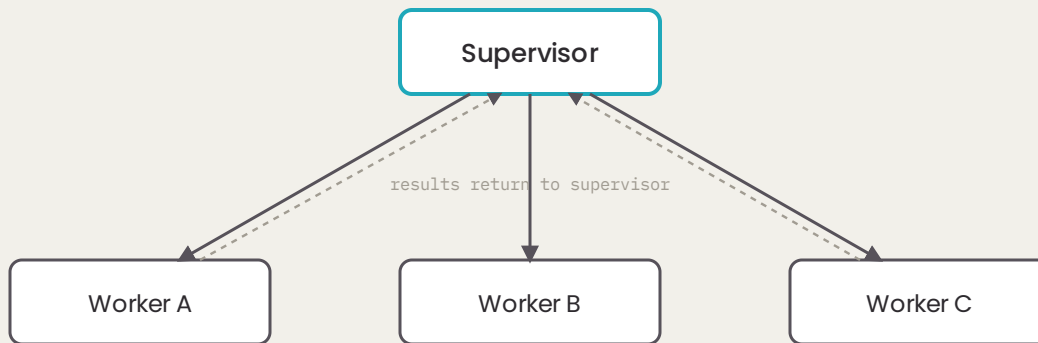
03

## The failure modes are the point

The diagram is the easy part. The "where it breaks" section is what a demo won't show you.

## 01 Supervisor / Worker

A supervisor agent reads the incoming task, decides which specialist can handle it, and routes accordingly. Workers don't talk to each other — everything passes back through the supervisor.



### WHEN TO USE THIS

- Tasks fall into a small number of clearly distinguishable categories.
- You want one place that owns the "what happens next" decision.
- Workers need to stay narrow and independently testable.

### WHERE IT BREAKS

- The supervisor becomes a bottleneck if every result must round-trip through it before the next step.
- Misrouting is silent — a task sent to the wrong worker often still returns *an* answer, just the wrong one.
- Adding workers without revisiting the routing logic degrades routing accuracy over time.

### IN PRACTICE

Sentinel's own `flag_injection` and `check_groundedness` tools are effectively two workers behind an implicit supervisor — the calling agent decides which check a given response needs.

## 02 Sequential Handoff

A fixed pipeline of specialist agents, each one doing a defined transformation and passing its output to the next. No branching, no routing decision — the order is the architecture.



### WHEN TO USE THIS

- The task genuinely decomposes into ordered stages, each needing different context or tools.
- Each stage's output is independently inspectable — good for debugging and partial re-runs.
- You want the simplest possible orchestration logic — there's no routing to get wrong.

### WHERE IT BREAKS

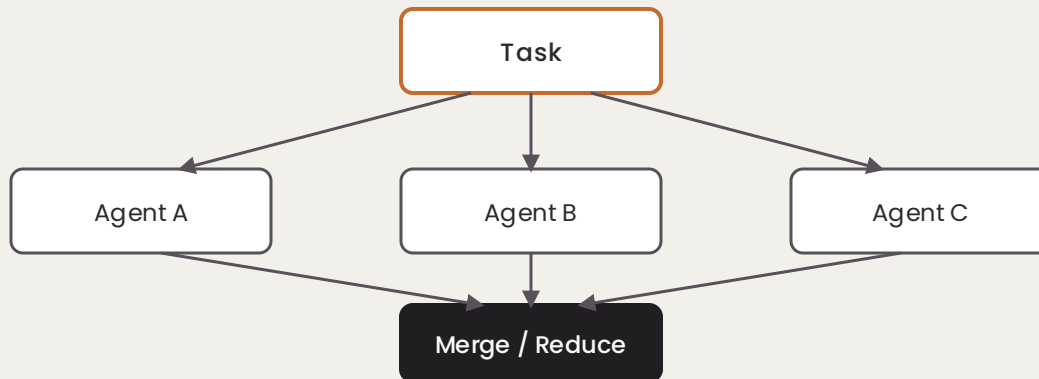
- A single stuck or wrong stage blocks everything after it — the whole pipeline is only as reliable as its weakest link.
- Errors compound silently: stage 3 rarely knows stage 1 already went subtly wrong.
- Rigid ordering can't handle a task that doesn't actually need every stage.

### IN PRACTICE

The RAG pipeline itself is a sequential handoff — retrieve, assemble context, generate, verify groundedness. See the [RAG Architecture Decision Template](#) for the tradeoffs at each stage.

## 03 Parallel Fan-Out / Fan-In

The same task is distributed to several agents at once — often the same agent run several times, or several agents each taking a different angle — then a merge step reconciles the results into one output.



### WHEN TO USE THIS

- Multiple independent perspectives improve the answer — self-consistency sampling, multi-query retrieval.
- The sub-tasks genuinely don't depend on each other's output.
- Latency budget allows parallel calls instead of paying their cost sequentially.

### WHERE IT BREAKS

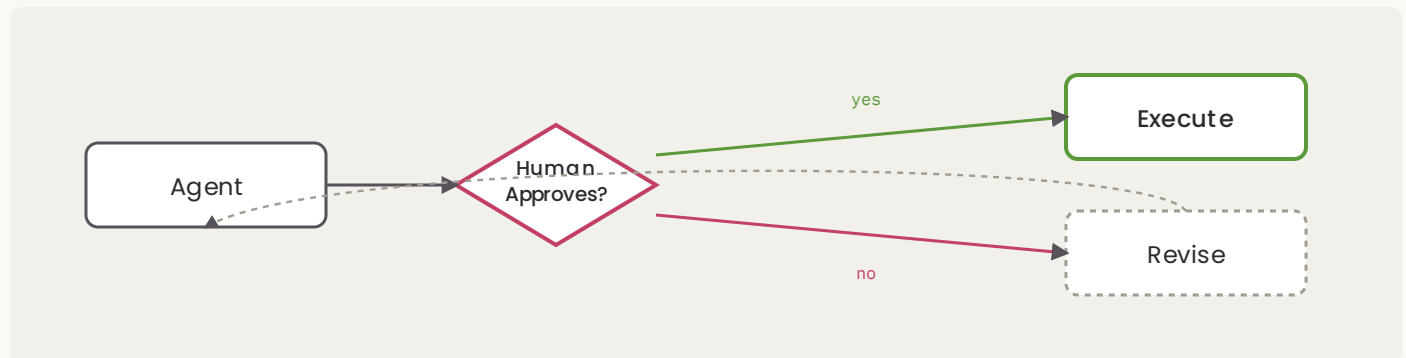
- The merge step is where the real complexity hides — reconciling disagreeing agents is its own hard problem.
- Cost multiplies by the fan-out factor; easy to lose track of what this does to a bill at scale.
- A single slow branch still holds up the merge — fan-out doesn't remove your tail-latency problem, it relocates it.

### IN PRACTICE

Sentinel's two metrics (injection, groundedness) can run fan-out in parallel on the same response — they're independent checks with no ordering dependency between them.

## 04 Human-in-the-Loop Approval Gate

An agent proposes an action but execution pauses at a defined checkpoint until a human approves, rejects, or edits it. The gate is a first-class step in the flow, not an afterthought bolted on.



### WHEN TO USE THIS

- The action is high-stakes, expensive to undo, or touches money, health, or legal outcomes.
- Model confidence is genuinely uncertain and a human can resolve ambiguity faster than more automation can.
- Regulatory or compliance requirements mandate a documented human sign-off.

### WHERE IT BREAKS

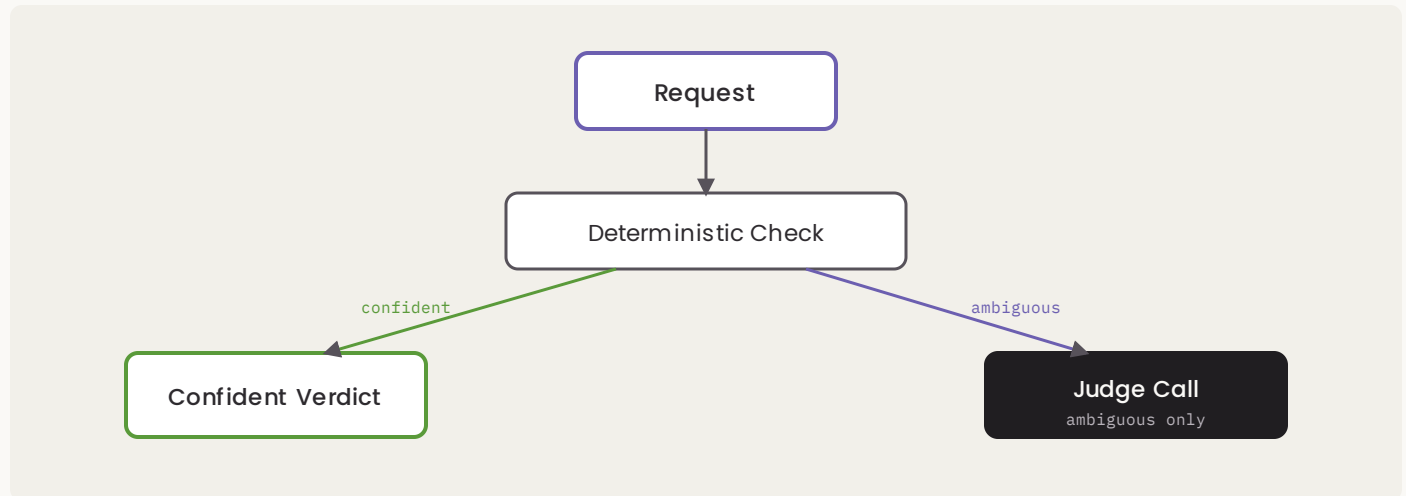
- Reviewer fatigue turns the gate into a rubber stamp if it fires too often on low-risk actions.
- No defined timeout or fallback path means an unavailable reviewer silently stalls the whole system.
- A gate with insufficient context handed to the reviewer produces approvals with no real judgment behind them.

### IN PRACTICE

Every `TenantPolicy` in Sentinel that sets `onTimeout: "fail_closed"` is implicitly choosing this pattern's fallback behavior — block rather than silently proceed when the gate can't be resolved.

## 05 Deterministic-First Escalation

Cheap, explainable checks try to resolve a request with high confidence first. Only the genuinely ambiguous middle escalates to a model call — keeping the common case fast, cheap, and auditable.



### WHEN TO USE THIS

- Most inputs are actually easy to classify correctly with simple, explainable logic.
- Cost or latency budget can't tolerate a model call on every single request.
- Confident decisions need to be auditable without pointing at an opaque model output.

### WHERE IT BREAKS

- Deterministic rules calibrated once quietly drift out of sync with real traffic as it changes.
- No timeout policy on the judge call turns "escalate when ambiguous" into "hang when ambiguous."
- Treating the fallback (fail-open vs. fail-closed) as a technical detail instead of a real product decision.

### IN PRACTICE

This is Sentinel's actual architecture — `flag_injection` and `check_groundedness` both use exactly this escalation ladder. See [mleg.tech/sentinel](https://mleg.tech/sentinel) for the working code.

# Patterns compose

None of these five are mutually exclusive, and most production systems end up as two or three nested together — a supervisor whose workers each run a sequential pipeline, with a human approval gate before the highest-stakes step, and deterministic-first escalation guarding the whole thing from unnecessary model calls. Start from the failure mode you're most worried about, not from whichever pattern is easiest to diagram.

```
// FOR PATTERN 5 SPECIFICALLY
```

## Sentinel — an eval-as-MCP-server trust layer

A working, free reference implementation of the deterministic-first escalation ladder — the exact pattern described on the previous page, running in production-shaped code.

[mleg.tech/sentinel](https://mleg.tech/sentinel)

If you're looking at an existing multi-agent system and can't confidently say which of these five patterns it's actually using — or it's clearly an accidental mix with no seam explicitly designed — that's usually the sign it's time for an architecture review rather than another feature added on top. If that's where you've landed, [reach out](#).